ThriveLens R0 Blueprint

Scope: Evidence-first Runtime Foundation / TL-R0-003

ThriveLens R0 Blueprint (Current Delivery Slice)

Status date: 12-August-2026

Repository: local `ThriveLens 2026` branch `codex/thrivelens-integration`

Canonical private mirror: `masterkush808/thrivelens-r0-postgres-foundation` (private)

1) Executive summary

ThriveLens R0 is a **local wellness platform foundation slice** focused on a safe, reversible, and evidence-driven backend/runtime proof path:

- A dedicated WSL2 Postgres runtime is prepared, validated, and cleaned deterministically.
- No public production API, mobile app, AI model serving, provider integrations, or full user domain persistence is active in R0.
- All critical runtime outcomes are proven by explicit scripts and checks, not by assumptions.
- The current blocker is **host memory headroom**, not application logic.

The key goal of this slice is a trustworthy execution surface for future app slices, not full product readiness.

---

2) What this slice is / is not

In scope (R0)

- Controlled PostgreSQL bootstrap and proof lifecycle inside a dedicated distro/worktree.
- Strict control-plane, lease, lock, and cleanup contracts.
- Runtime budget and resource accounting gates.
- Security/privacy hardening around raw output, process capture, and state transitions.
- Evidence-first release discipline (scripts/tests/reports).

Explicitly out of scope in R0

- Real user accounts, nutrition/pose/exercise production workflows.
- Paid providers, real-world billing, migration/deployment to production infrastructure.
- Android tooling, heavy mobile/web runtime bundling, Docker-heavy parallel execution.
- Local model training or checkpoint-heavy AI inference.
- Any clinical or prescription behavior.

---

3) Core architecture

Design style

- Script-first, idempotent control flow.
- Immutable task provenance via command logs and structured result contracts.
- Guarded authority objects:

- **lifecycle lock**

- **configuration lease**

- **distro identity token**

- **contract fingerprint**

- Bounded operations, fail-closed errors, and explicit cleanup semantics.

Primary components

1. **start adapter (`scripts/dev/postgres/start.ps1`)**

- Coordinates the runtime command sequence and final result status.

2. **runtime module (`scripts/dev/postgres/Runtime.psm1`)**

- Executes bounded resource accounting subprocesses and strict result parsing.

3. **WslRuntime module (`scripts/dev/postgres/WslRuntime.psm1`)**

- Performs exact host/distro operations and cleanup verification.

4. **preflight (`scripts/dev/postgres/preflight.ps1`)**

- Admission-like runtime check before mutation.

5. **control tests (`test_runtime.ps1`, `test_wsl_controls.ps1`, plus static/security tests)**

- Validate ordering, timeout/capture semantics, and malformed/mutated branches.

6. **detached execution path**

- Sequence runner/broker/launcher scripts in `%LOCALAPPDATA%\ThriveLens\runtime-captures\TL-R0-003` for safer execution.

---

4) Run contract used for command 3

The canonical proof command stack is:

1. `pwsh -NoProfile -File scripts/check_resource_budget.ps1`

2. `pwsh -NoProfile -File scripts/dev/postgres/preflight.ps1`

3. `pwsh -NoProfile -File scripts/dev/postgres/test_runtime.ps1`

4. `pwsh -NoProfile -File scripts/check_resource_budget.ps1`

All four must succeed for a full TL-R0 external runtime proof.

Current observed blocker: command-3 commonly exits BLOCKED with

`LOW_FREE_MEMORY_AFTER_WSL_PROBES` when host headroom cannot sustain the reclaim + probe sequence. This is a **resource envelope problem**, not a crash in control logic.

---

5) Memory and stability rationale

Why it currently “hiccups”

- WSL command bursts and verification probes are intentionally strict and consume RAM

temporarily.

- On low free-memory systems, command 3 can block on reclaim gates even after preflight passed.
- The system currently keeps strict limits to avoid unsafe continuation; that means it can fail fast instead of over-using host memory.

Practical best practice

- Keep the host idle (especially editor/runtime helpers) during a run.
- Run via detached runner to decouple from IDE process-tree volatility.
- Run only with sufficient free RAM (practically: room above the reclaim threshold plus normal activity buffer).

---

6) Error/decision model (high level)

- Every major step is explicit and mapped to bounded public codes.
- Failure paths are intentionally conservative:

- `BLOCKED` for recoverable environmental gating failures.

- `ERROR` for cleanup/authority/containment failure.

- Cleanup and release counters are preserved so proof data remains auditable even on failures.

---

7) Security and privacy posture (R0)

- No raw stdout/stderr leak through final machine-readable parser paths.
- Strict JSON schema checks with fail-closed handling.
- Process capture is bounded and drained with timeout semantics.
- No automatic retries on uncertain post-run states without explicit recovery path.
- No model/provider user data ingestion in R0.

Security policy decisions (e.g., dataset/model licensing for future slices) are tracked in program docs and remain pending in controlled release states.

---

8) Current performance constraints

- No multi-GB model checkpoint downloads in ordinary bootstrap.
- Sequenced, bounded builds/checks to stay within constrained hardware.
- No concurrent heavy tasks (Android emulator + database + Flutter/web builds).
- Current constraint: headroom-sensitive command 3 reclaim gate.

---

9) Repository and governance map

- **Control plane:** `docs/program/*`, `scripts/dev/postgres/*`, `scripts/check_resource_budget.ps1`.
- **Validation:** `scripts/test_control_plane_validator.py`, runtime static/security tests.
- **Evidence:** run reports and review docs under `docs/program/evidence/*` (outside this document generation scope).

The project uses strict release-state gates; evidence and task ownership must stay coherent before claiming VERIFIED.

---

10) About a lighter model (direct answer)

You asked about a lighter model to reduce RAM:

- **In this R0 slice, there is no active local large model inference workload**.
- The high RAM behavior comes from the WSL/Postgres verification path, not an active model runner.
- If/when a model layer is added in a later slice, lighter alternatives are feasible:

- small-context API models (off-host),

- quantized local models,

- strict budget caps and memory guardrails.

So the practical immediate fix is not model swap here; it is **resource orchestration + detached, host-threshold-friendly execution**.

---

11) What I completed for you now

1. Confirmed/verified private GitHub migration under

`masterkush808/thrivelens-r0-postgres-foundation`.

2. Drafting this blueprint for your review.

3. Generated a direct, printable export path for the blueprint PDF.

---

12) How to run this cleanly to avoid IDE-related hiccups

- Prefer running the detached command chain from a stable terminal/shell session.
- Avoid launching inside an already RAM-heavy IDE context.
- Keep VS Code/Codex as secondary monitor, not the only runtime parent.
- Re-run only when free memory is visibly above reclaim threshold margin.

---

13) Recommended next step

If you want, I'll now produce the **single next run plan** with exact shell commands for:

1) pre-run memory wait strategy,

2) detached execution sequence,

3) evidence file collection,

4) failure-state interpretation in plain language.